

This list comprehension contains two `for` expressions, but it's difficult for even experienced Python developers to understand. The expanded form, which uses two `for` loops, creates the same flattened list but is much easier to read:

```
>>> nestedList = [[0, 1, 2, 3], [4], [5, 6], [7, 8, 9]]
>>> flatList = []
>>> for sublist in nestedList:
...     for num in sublist:
...         flatList.append(num)
...
>>> flatList
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Comprehensions are syntactic shortcuts that can produce concise code, but don't go overboard and nest them within each other.

Empty except Blocks and Poor Error Messages

Catching exceptions is one of the primary ways to ensure that your programs will continue to function even when problems arise. When an exception is raised but there is no `except` block to handle it, the Python program crashes by immediately stopping. This could result in losing your unsaved work or leaving files in a half-finished state.

You can prevent crashes by supplying an `except` block that contains code for handling the error. But it can be difficult to decide how to handle an error, and programmers can be tempted to simply leave the `except` block blank with a `pass` statement. For example, in the following code we use `pass` to create an `except` block that does nothing:

```
>>> try:
...     num = input('Enter a number: ')
...     num = int(num)
... except ValueError:
...     pass
...
Enter a number: forty two
```
